
Artificial Intelligence

Tut/Lab week 2

COSC2129

Outline

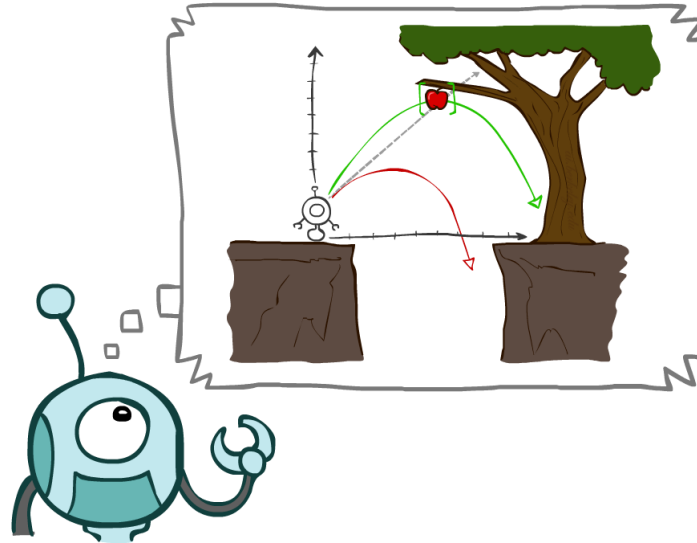
- Search
 - Blind search
 - Informed search
 - Hands-on examples
- Assignment 1 part 1
- Q&A

Course material

- Book: [Artificial Intelligence: A Modern Approach](#)
- Book - [TOC of Artificial Intelligence: A Modern Approach, 4th Global ed.](#)
- AIMA: [Pseudo code](#)
- [Good practices Python](#)

Activity 1

- What is Search?
 - Search is an exploration of possibilities

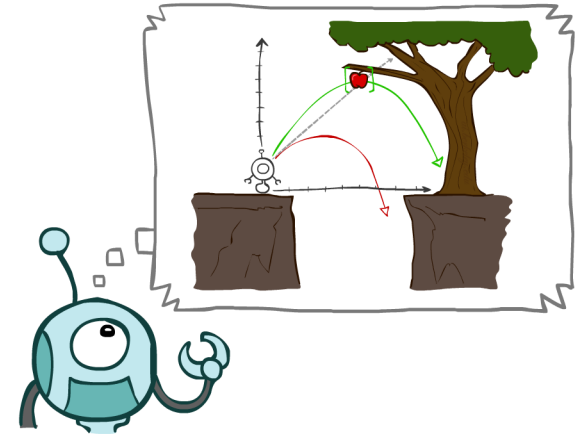
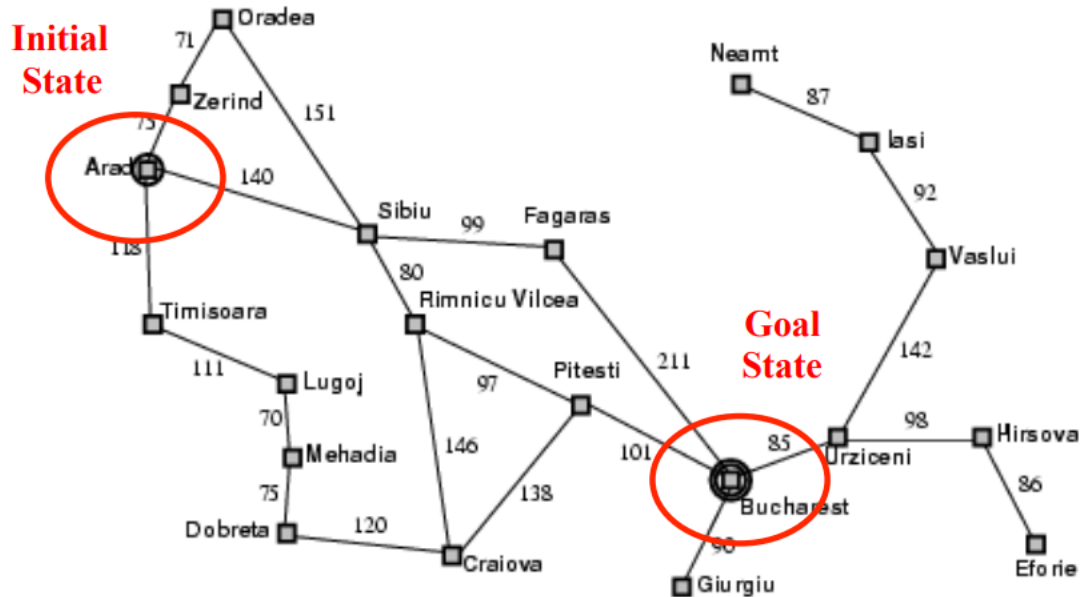


Activity 1

- Give some examples of applications of problem solving by search.

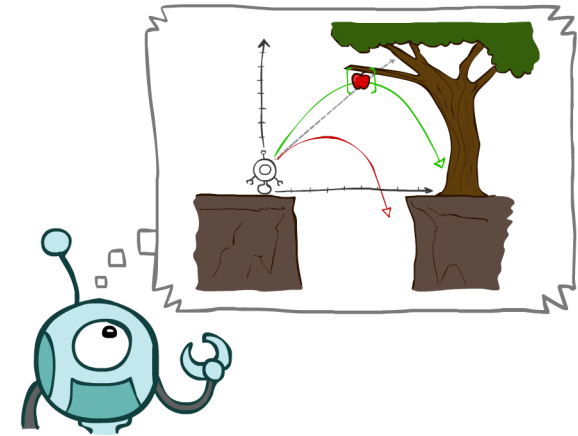
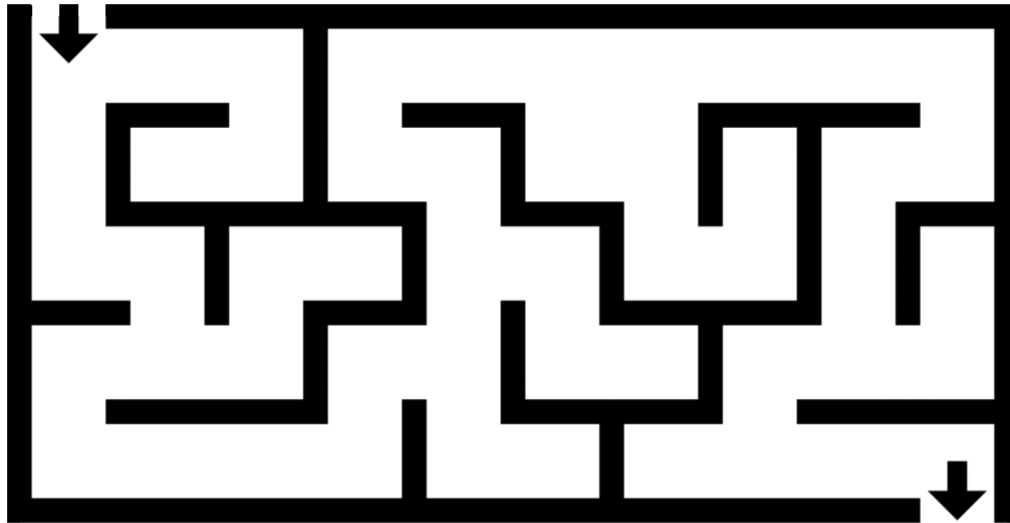
Why search?

- Path finding problem



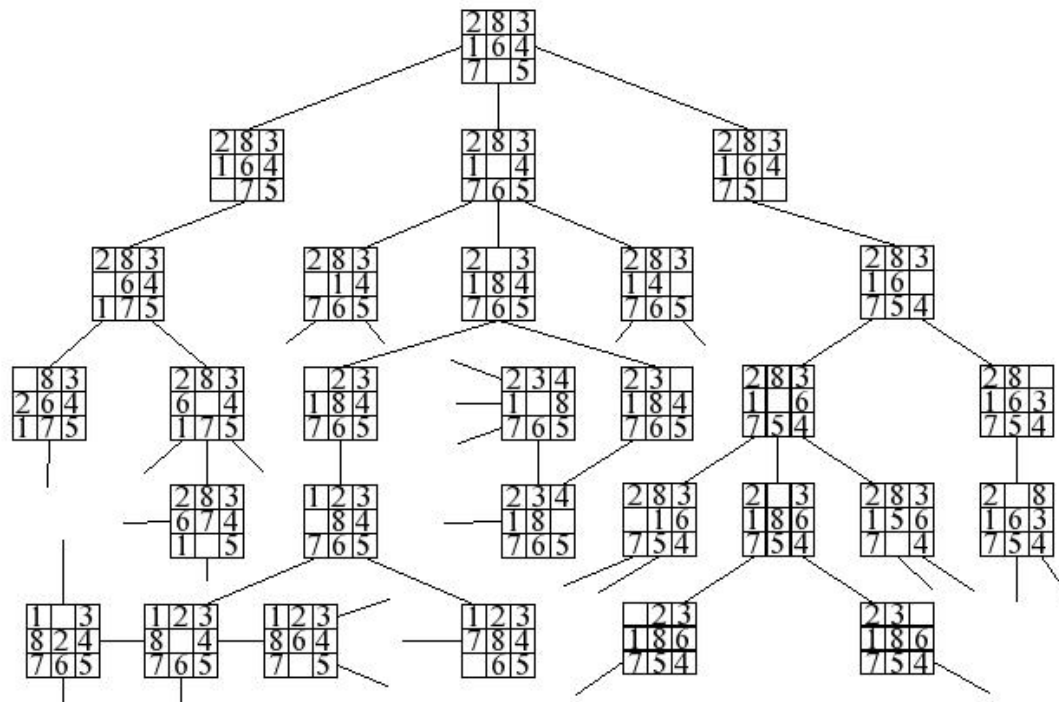
Why search?

- Path finding problem



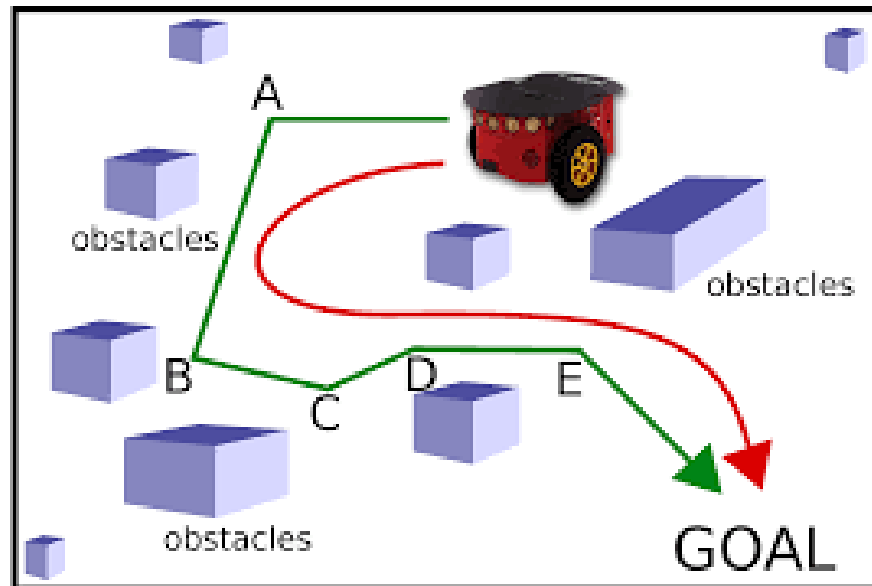
Why search

- 8 puzzle problem



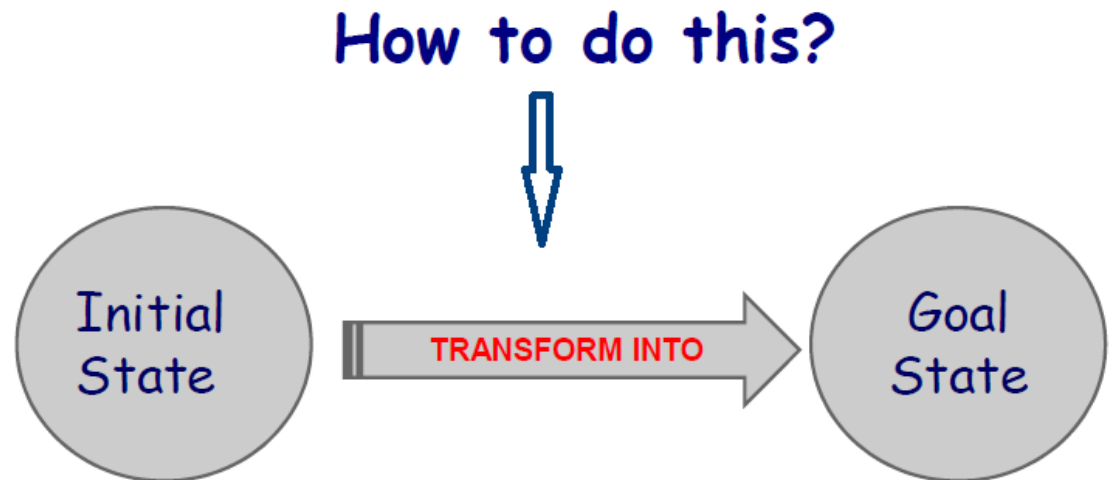
	1	2
3	4	5
6	7	8

Problem solving by search



Search Problems

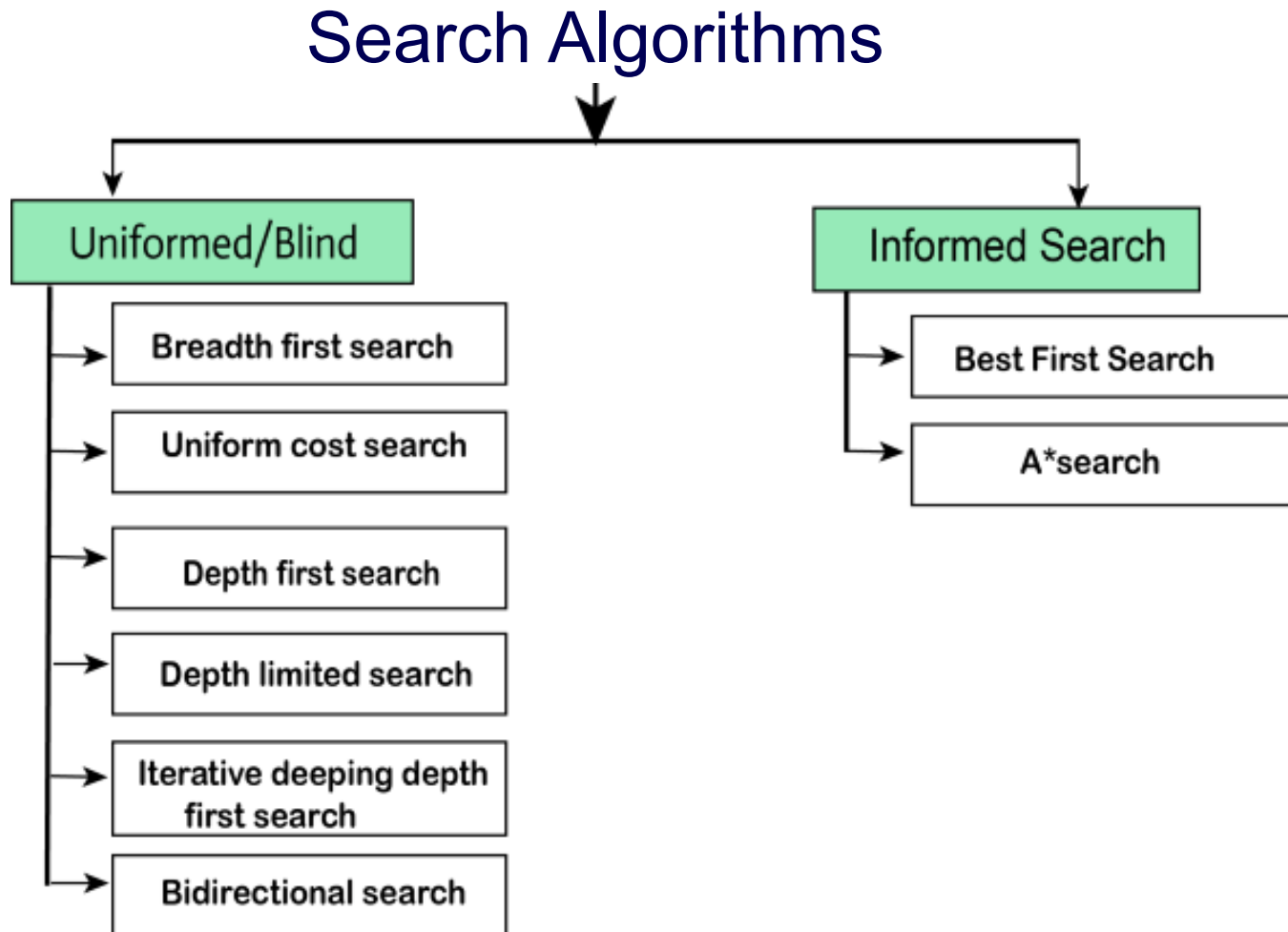
- A search problem is defined by:
 - Possible states (search space)
 - Initial state
 - Actions
 - Transition mode
 - Goal test
 - Path cost



Activity 2

- What is a state for each of the example above?
- How to represent a state?
- What is a state space?
- How to represent it?

Search Algorithms



Performance of Search Strategies

- **Completeness:** Is it guaranteed to find a solution if one exists?
- **Optimality:** Will it find an optimal solution?
- **Time complexity:** How many nodes get expanded?
- **Space complexity:** How much memory is needed?
- **Time and space complexity is calculated based on:**
 - b : Branching factor
 - d : Depth of the shallowest goal node
 - m : Maximum length of any path in the state space

Activity 3: Hand-on Practice

- Hand-on slides

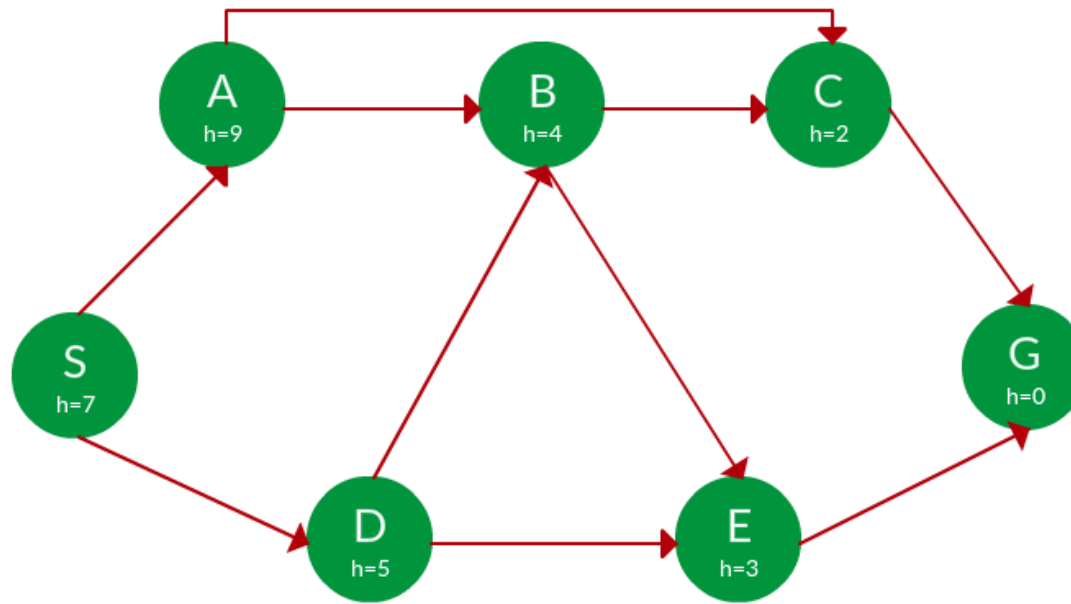
Practice

- Recap Greedy Search: Use heuristics to guide the search
- Heuristic:
 - A heuristic h is defined as $h(x) = \mathbf{Estimate}$ of distance of node x from the goal node.
 - Lower the value of $h(x)$, closer is the node from the goal.
- Strategy: Expand the node closest to the goal state, i.e. expand the node with *a lower h value*.

Practice

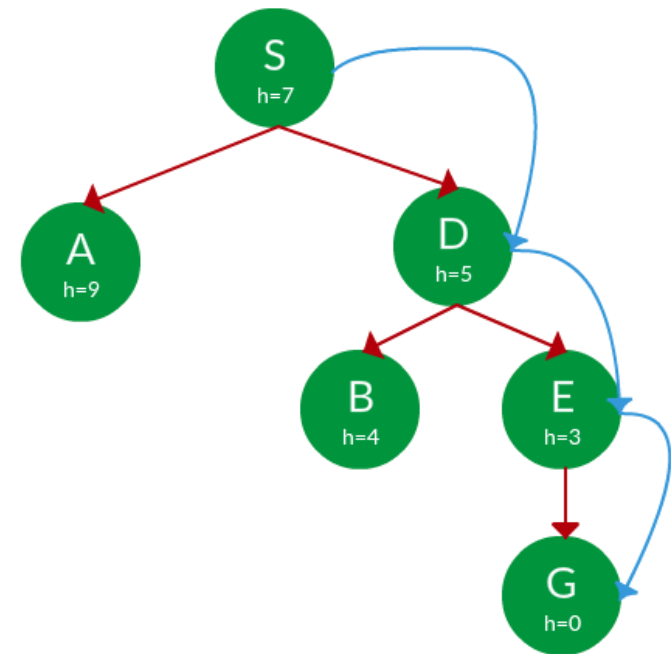
- Greedy Search example

Find the path from S to G using greedy search. The heuristic values h of each node below the name of the node.



Practice

- Solution
 - Start from S, we can traverse to A(h=9) or D(h=5). We choose D, as it has the lower heuristic cost.
 - From D, we can move to B(h=4) or E(h=3). We choose E with a lower heuristic cost.
 - From E, we go to G(h=0). This entire traversal is shown in the search tree below.



Practice

- A* Tree Search

- Heuristic: $f(x) = g(x) + h(x)$

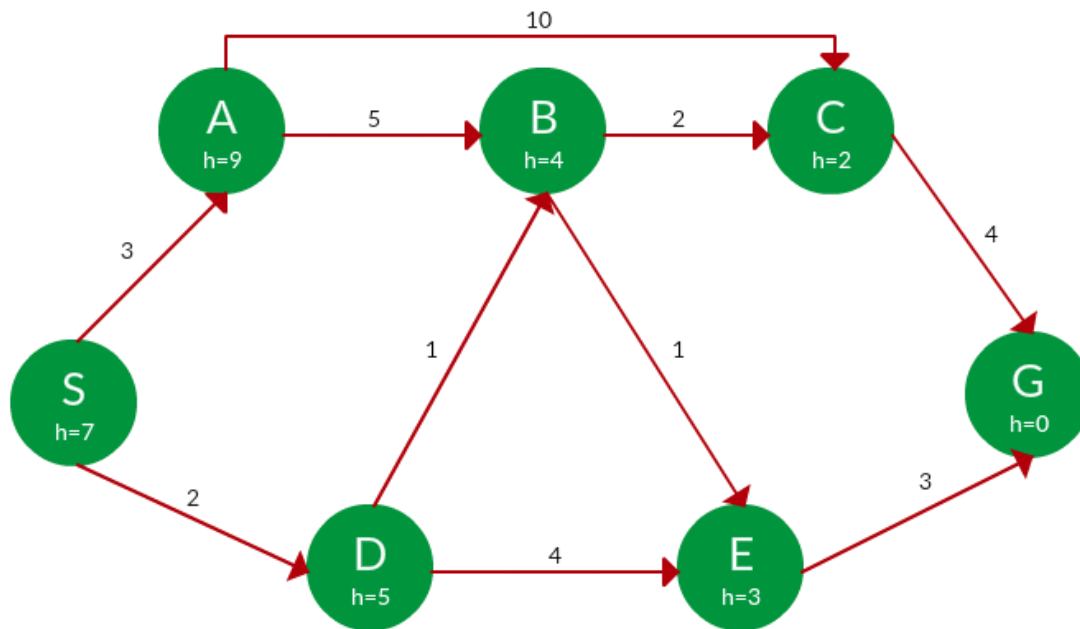
- Here, $h(x)$, *the forward cost*, is an estimate of the distance of the current node from the goal node.
 - And, $g(x)$, *the backward cost*, is the cumulative cost of a node from the root node.
 - Admissibility:

$$0 \leq h(x) \leq h^*(x)$$

- **Strategy:** Choose the node with the lowest $f(x)$ value.

Practice

- A* search: $f(x) = g(x) + h(x)$



Practice

- Solution

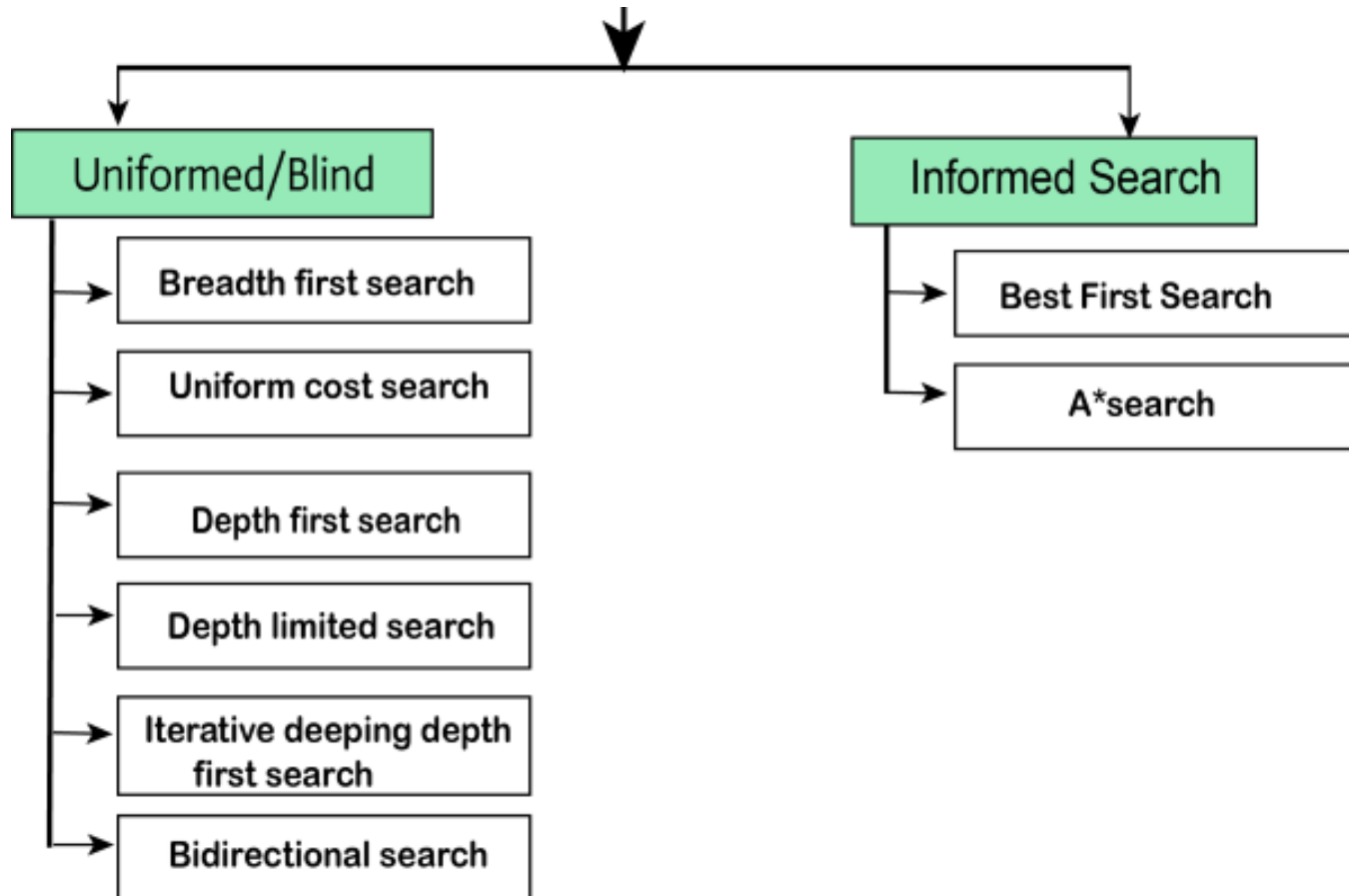
Path		$h(x)$	$g(x)$	$f(x)$
S		7	0	7
S -> A		9	3	12
S -> D	✓	5	2	7
S -> D -> B	✓	4	$2 + 1 = 3$	7
S -> D -> E		3	$2 + 4 = 6$	9
S -> D -> B -> C	✓	2	$3 + 2 = 5$	7
S -> D -> B -> E	✓	3	$3 + 1 = 4$	7
S -> D -> B -> C -> G		0	$5 + 4 = 9$	9
S -> D -> B -> E -> G	✓	0	$4 + 3 = 7$	7

Path: S -> D -> B -> E -> G

Cost: 7

Recap

Search Algorithms



Project 0

- CS188 Intro to AI - Course Materials
- Tutorial: <http://ai.berkeley.edu/tutorial.html>

Activity 4

- PACMAN and Assignment 1 part 1

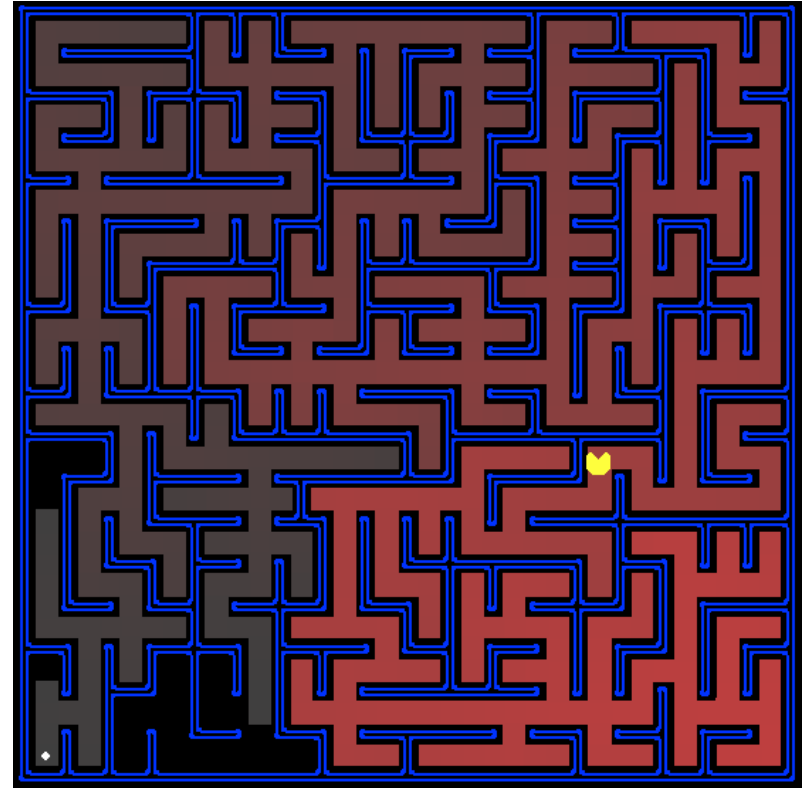


The game remains one of the highest-grossing and best-selling games, generating more than \$14 billion in revenue (as of 2016) and 43 million units in sales combined, and has an enduring commercial and cultural legacy, commonly listed as one of the [greatest video games of all time](#).

<https://en.wikipedia.org/wiki/Pac-Man>

Assignment 1-1: Pacman game

- Pacman in A1-1
 - Question 1
 - Question 2
 - Question 3
 - Question 4
 - Question 5
 - Question 6
 - Question 7
 - Question 8



<http://ai.berkeley.edu/search.html>

Notes

Files you'll edit:

[search.py](#) Where all of your search algorithms will reside.

[searchAgents.py](#) Where all of your search-based agents will reside.

Files you might want to look at:

[pacman.py](#) The main file that runs Pacman games. This file describes a Pacman GameState type, which you use in this project.

[game.py](#) The logic behind how the Pacman world works. This file describes several supporting types like AgentState, Agent, Direction, and Grid.

[util.py](#) Useful data structures for implementing search algorithms.

Supporting files you can ignore:

[graphicsDisplay.py](#) Graphics for Pacman

[graphicsUtils.py](#) Support for Pacman graphics

[textDisplay.py](#) ASCII graphics for Pacman

[ghostAgents.py](#) Agents to control ghosts

[keyboardAgents.py](#) Keyboard interfaces to control Pacman

Notes

- **Hint:** Each algorithm is very similar. Algorithms for DFS, BFS, UCS, and A* differ only in the details of how the fringe is managed.
 - Concentrate on DFS, the rest should be relatively straightforward.
 - Implement the depth-first search (DFS) algorithm in the `depthFirstSearch` function in `search.py`.
 - To make your algorithm complete, write the graph search version of DFS, which avoids expanding any already visited states.

Question?